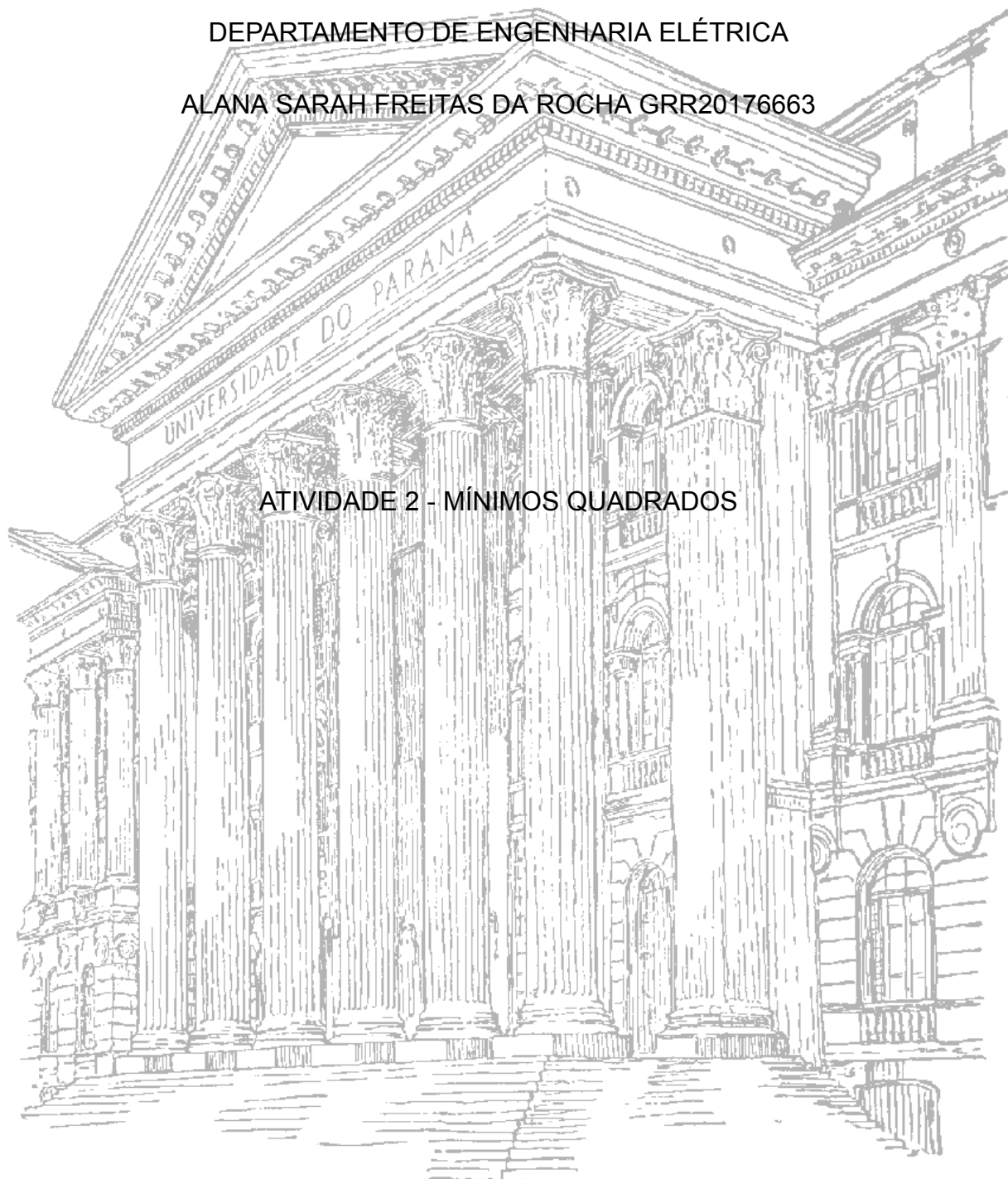


UNIVERSIDADE FEDERAL DO PARANÁ  
DEPARTAMENTO DE ENGENHARIA ELÉTRICA  
ALANA SARAH FREITAS DA ROCHA GRR20176663



CURITIBA  
2024

## SUMÁRIO

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>1.INTRODUÇÃO</b>                                  | <b>3</b>  |
| <b>2. FUNDAMENTAÇÃO TEÓRICA</b>                      | <b>3</b>  |
| <b>2.1 MODELO POLINOMIAL COM MEMÓRIA</b>             | <b>3</b>  |
| <b>2.3 ENUNCIADO DA ATIVIDADE</b>                    | <b>3</b>  |
| <b>2.4 RESOLUÇÃO MANUAL PARA PRIMEIROS INSTANTES</b> | <b>4</b>  |
| <b>3.RESOLUÇÃO UTILIZANDO PYTHON</b>                 | <b>6</b>  |
| <b>4 CONCLUSÃO</b>                                   | <b>9</b>  |
| <b>5.REFERÊNCIAS</b>                                 | <b>10</b> |

## 1.INTRODUÇÃO

Nesta atividade vamos criar um modelo matemático para um amplificador, a partir de 99 dados de entrada e saída, para isso vamos utilizar o modelo polinomial com memória. Devido ao alto número de dados, utilizaremos Python para criar a matriz de regressão  $XX$ , além disso o ajuste dos coeficientes será realizado a partir do método dos mínimos quadrados a ser calculado no Python, conforme já apresentado na última atividade.

## 2. FUNDAMENTAÇÃO TEÓRICA

Utilizaremos o modelo polinomial com memória para criação de um modelo matemático para um amplificador, pois ele permite capturar a complexidade do comportamento desses dispositivos que apresentam não linearidade e efeitos de memória significativos. Ao considerar potências da entrada atual e passada, o modelo é capaz de descrever distorções não lineares e dependências dinâmicas, tornando-o uma escolha adequada para modelar amplificadores não ideais e sistemas dinâmicos complexos.

### 2.1 MODELO POLINOMIAL COM MEMÓRIA

O modelo polinomial com memória é uma simplificação da série de Volterra, empregada na modelagem de sistemas onde a saída no tempo  $n$  depende do valor atual da entrada quanto de valores anteriores da memória, considerando a não linearidade por meio de termos polinomiais. Basicamente esse modelo assume que a saída  $y(n)$  resulta da combinação de diversas potências dos valores presentes e passados da entrada  $u(n)$ .

A fórmula geral do modelo é:

$$y(n) = \sum_{m=0}^{M-1} \sum_{p=1}^P a_{m,p} \cdot u(n-m)^p$$

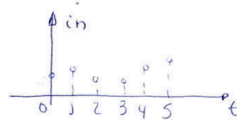
Onde:

- $y(n)$  é a saída no instante  $n$ ,
- $u(n-m)$  são os valores da entrada em instantes passados,
- $P$  é a ordem do polinômio, que indica o grau da não linearidade,
- $a_{m,p}$  são os coeficientes que definem o peso de cada termo.

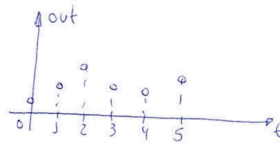
### 2.3 ENUNCIADO DA ATIVIDADE

Deseja-se criar o modelo matemático para um amplificador utilizando o polinômio com memória:

EXERCÍCIO: CONSIDERE UM CONJUNTO DE DADOS DE ENTRADA E SAÍDA DE UM AMPLIFICADOR:

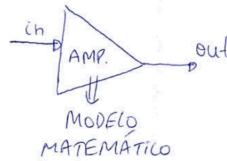


$$in = \begin{bmatrix} in(0) \\ in(1) \\ in(2) \\ \vdots \\ in(99) \end{bmatrix}$$



$$out = \begin{bmatrix} out(0) \\ out(1) \\ out(2) \\ \vdots \\ out(99) \end{bmatrix}$$

DESEJA-SE CRIAR UM MODELO MATEMÁTICO P/ O AMPLIFICADOR.



O MODELO ESCOLHIDO É O POLINÔMIO COM MEMÓRIA, UM CASO PARTICULAR DA SÉRIE DE VOLTERRA.

$$d(n) = out(n) = \sum_{p=1}^P \sum_{m=0}^M h_p(m) [in(n-m)]^p = X_{in}^T H(n)$$

## 2.4 RESOLUÇÃO MANUAL PARA PRIMEIROS INSTANTES

Modelo polinomial com memória, é expresso por:

$$y(m) = \sum_{m=0}^{M-1} \sum_{p=1}^P a_{m,p} \cdot u(m-m)^p$$

$y(m)$  = saída  
 $u(m-m)$  = entrada nos diferentes valores passados  
 $P=2$  (ordem do polinômio)  
 $M=2$  memória, (consideramos valores atuais  $u(m)$  e anterior  $u(m-1)$ )  
 $a_{m,p}$  = coeficientes que precisamos determinar

Dados fornecidos:

$u$  = entrada  $u(m)$   
 $in: [0.62, 0.60, 0.62, 0.66, 0.69]$   
 $out: [0.49, 0.47, 0.47, 0.48, 0.54]$   
 $y$  = saída  $y(m)$

Parâmetros considerados:

$P=2$  (ordem 2, incluindo até  $u^2$ )  
 $M=2$  (considerando até 2 últimos instantes)

A fórmula do modelo será

$$y(m) = a_{0,1} \cdot u(m) + a_{0,2} \cdot u(m)^2 + a_{1,1} \cdot u(m-1) + a_{1,2} \cdot u(m-1)^2$$

A seguir iremos criar  $y$  e  $xx$  para os primeiros instantes

$$\text{Para } m=2: y(2) = a_{0,1} \cdot u(2) + a_{0,2} \cdot u(2)^2 + a_{1,1} \cdot u(1) + a_{1,2} \cdot u(1)^2$$

Usando os dados:

$$u(2) = 0.62$$

$$u(1) = 0.60$$

$$\text{temos } y(2) = a_{0,1} \cdot 0.62 + a_{0,2} \cdot (0.62)^2 + a_{1,1} \cdot 0.60 + a_{1,2} \cdot (0.60)^2$$

$$\text{Para } m=3: y(3) = a_{0,1} \cdot u(3) + a_{0,2} \cdot u(3)^2 + a_{1,1} \cdot u(2) + a_{1,2} \cdot u(2)^2$$

$$u(3) = 0.66$$

$$u(2) = 0.62$$

Analogamente:

$$y(3) = a_{0,1} \cdot 0.66 + a_{0,2} \cdot (0.66)^2 + a_{1,1} \cdot 0.62 + a_{1,2} \cdot (0.62)^2$$

Para  $m=4$   $y(4) = a_{0,1} \cdot u(4) + a_{0,2} u(4)^2 + a_{1,1} u(3) + a_{1,2} \cdot u(3)^2$   
 $u(4) = 0,69$   
 $u(3) = 0,66$

Assim:  $y(4) = a_{0,1} \cdot 0,69 + a_{0,2} \cdot (0,69)^2 + a_{1,1} \cdot 0,66 + a_{1,2} (0,66)^2$

Vetor  $y$  contém as saídas, assim:

$$y = \begin{bmatrix} y(2) \\ y(3) \\ y(4) \end{bmatrix} = \begin{bmatrix} 0,47 \\ 0,48 \\ 0,50 \end{bmatrix}$$

Matriz  $XX$  correspondente aos termos  $u(m), u(m)^2, u(m-1), u(m-1)^2$

Assim temos:

$$XX(2) = \begin{bmatrix} u(2) & u(2)^2 & u(1) & u(1)^2 \end{bmatrix} = \begin{bmatrix} 0,62 & (0,62)^2 & 0,60 & 0,60^2 \end{bmatrix}$$

$$XX(3) = \begin{bmatrix} u(3) & u(3)^2 & u(2) & u(2)^2 \end{bmatrix} = \begin{bmatrix} 0,66 & (0,66)^2 & 0,62 & 0,62^2 \end{bmatrix}$$

$$XX(4) = \begin{bmatrix} u(4) & u(4)^2 & u(3) & u(3)^2 \end{bmatrix} = \begin{bmatrix} 0,69 & (0,69)^2 & 0,66 & 0,66^2 \end{bmatrix}$$

• Por conta disso podemos criar uma lógica no Python usando for (recursividade) para criar a matriz  $XX$ .

• Para calcular os coeficientes  $a$ , usamos a fórmula dos mínimos quadrados

$$a = (XX^T XX)^{-1} XX^T y$$

Onde:

$XX^T$  é a transposta de  $XX$   
 $(XX^T XX)^{-1}$  é a inversa  $XX^T XX$   
 $y$  é o vetor de saídas

Essa fórmula e resolução em python já foi apresentada na última atividade.

### 3.RESOLUÇÃO UTILIZANDO PYTHON

Podemos fazer a criação da matriz de regressão de maneira mais simples utilizando loop no Python:

```
# Definir a ordem do polinômio (P) e o número de atrasos (M)
P = 2 # Ordem do polinômio
M = 2 # Memória

# Número de dados
N = len(u) #99
print(N)
# Inicializar a matriz de regressão XX
XX = []

# Construir a matriz de regressão XX utilizando loop
for n in range(M, N): # Evitar os primeiros M valores, pois não teremos memória
    linha = []
    for m in range(M): # Para memória
        for p in range(1, P + 1): # Para cada potência
            linha.append(u[n - m] ** p)
    XX.append(linha)

# Converter a lista para um array NumPy
XX = np.array(XX)

# Ajustar o vetor de saída correspondente
y_adjusted = y[M:]

# Ver as primeiras linhas da matriz XX e do vetor Y
print("Primeiras linhas de XX:", XX[:5])
print("Primeiras linhas de Y:", y_adjusted[:5])
```

✓ 0.0s

```
99
Primeiras linhas de XX: [[0.62319308 0.38836962 0.60603586 0.36727947]
 [0.66196767 0.4382012 0.62319308 0.38836962]
 [0.69947785 0.48926926 0.66196767 0.4382012 ]
 [0.72012176 0.51857534 0.69947785 0.48926926]
 [0.71870468 0.51653642 0.72012176 0.51857534]]
Primeiras linhas de Y: [0.47848198 0.48890297 0.50141688 0.51037182 0.5128654 ]
```

Como apresentado na última atividade, a biblioteca NumPy disponibiliza uma função eficiente para resolver sistemas de equações lineares através dos mínimos quadrados: `linalg.lstsq`. Esta função retorna os coeficientes que minimizam o erro quadrático médio.

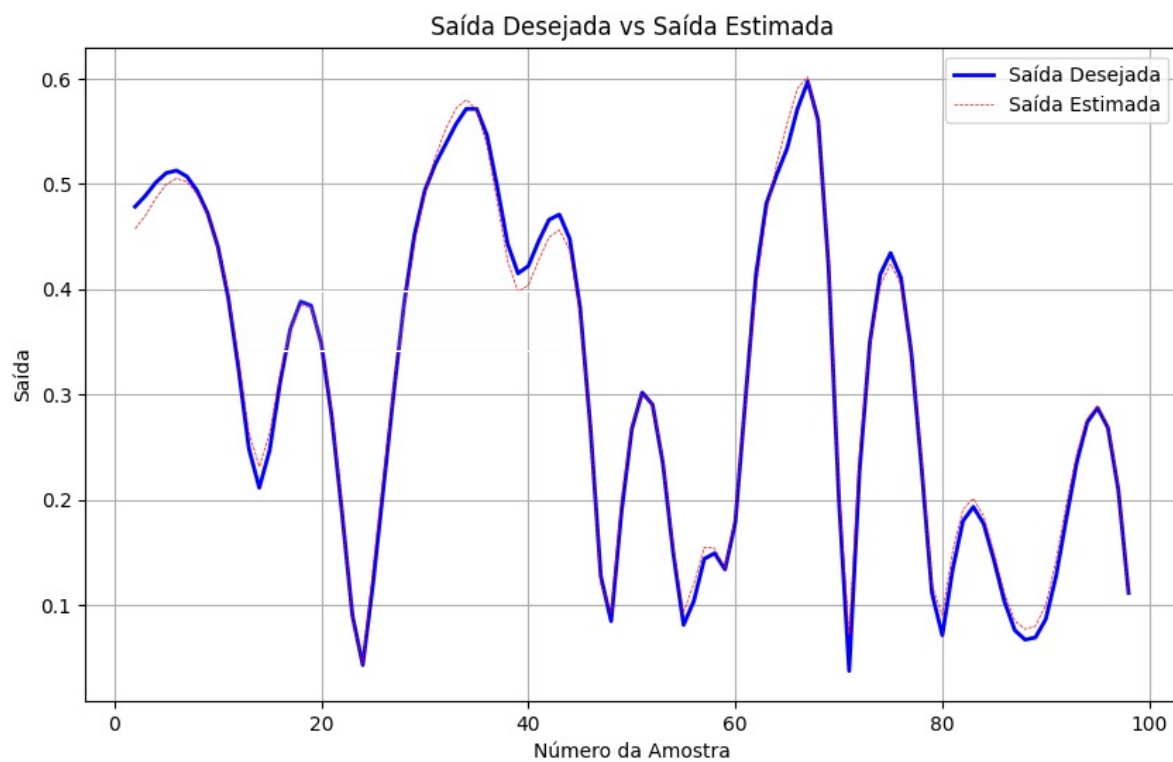
Aplicação do código:

```
# Resolver o problema de mínimos quadrados
coeffs = np.linalg.lstsq(XX, y_adjusted, rcond=None)[0]
# Exibir os coeficientes estimados
print("Coeficientes estimados:", coeffs)
```

4] ✓ 0.0s

```
Coeficientes estimados: [ 1.03625919 -0.63794799 -0.04639047 0.2382853 ]
```

Gráfico gerado:



Código Inteiro ([disponível no github](#)):

```
TCC_2.ipynb • main.ipynb
TCC_2.ipynb > # Resolver o problema de mínimos quadrados
+ Code + Markdown | ▶ Run All ⌂ Restart ⌂ Clear All Outputs | 📄 Variables 📄 Outline ...

import scipy.io
import numpy as np
# Carregar o arquivo .mat
data = scipy.io.loadmat('/Users/alanasarahfreitasdarocho/Documents/UFPR 8 periodo/Atividade 2 - TCC/IN_OUT_PA.mat')

# Exibir os cabeçalhos
print(data.keys())
u = data['in'].flatten() # Entrada, flatten é usado para transformar em uma unica dimensão
y = data['out'].flatten() # Saída

print(u[:5]) # printando 5 primeiros valores
print(y[:5])

[10] ✓ 0.0s
... dict_keys(['__header__', '__version__', '__globals__', 'in', 'out'])
[0.62302363 0.60603586 0.62319308 0.66196767 0.69947785]
[0.49243503 0.47870008 0.47848198 0.48890297 0.50141688]

# Definir a ordem do polinômio (P) e o número de atrasos (M)
P = 2 # Ordem do polinômio
M = 2 # Memória

# Número de dados
N = len(u) #99
print(N)
# Inicializar a matriz de regressão XX
XX = []

# Construir a matriz de regressão XX utilizando loop
for n in range(M, N): # Evitar os primeiros M valores, pois não teremos memória
    linha = []
    for m in range(M): # Para memória
        for p in range(1, P + 1): # Para cada potência
            linha.append(u[n - m] ** p)
    XX.append(linha)

# Converter a lista para um array NumPy
XX = np.array(XX)

# Ajustar o vetor de saída correspondente
y_adjusted = y[M:]

# Ver as primeiras linhas da matriz XX e do vetor Y
print("Primeiras linhas de XX:", XX[:5])
print("Primeiras linhas de Y:", y_adjusted[:5])

[19] ✓ 0.0s
... 99
Primeiras linhas de XX: [[0.62319308 0.38836962 0.60603586 0.36727947]
[0.66196767 0.4382012 0.62319308 0.38836962]
[0.69947785 0.48926926 0.66196767 0.4382012 ]
[0.72012176 0.51857534 0.69947785 0.48926926]
[0.71870468 0.51653642 0.72012176 0.51857534]]
Primeiras linhas de Y: [0.47848198 0.48890297 0.50141688 0.51037182 0.5128654 ]

▶ ~
# Resolver o problema de mínimos quadrados
coeffs = np.linalg.lstsq(XX, y_adjusted, rcond=None)[0]
# Exibir os coeficientes estimados
print("Coeficientes estimados:", coeffs)

[14] ✓ 0.0s
... Coeficientes estimados: [ 1.03625919 -0.63794799 -0.04639047 0.2382853 ]
```



## 4 CONCLUSÃO

Nesta atividade, criamos um modelo matemático polinomial com memória para ilustrar o funcionamento de um amplificador, empregando 99 dados de entrada e saída. O modelo possibilita a captura da não linearidade e dos efeitos de memória presentes no amplificador, oferecendo uma representação mais precisa e sólida do sistema dinâmico.

Usando o Python como ferramenta principal para gerenciar o volume de dados e a complexidade do modelo, construímos a matriz de regressão  $XX$ , que simboliza as entradas presentes e passadas, elevadas a variadas potências. O método dos mínimos quadrados foi utilizado para ajustar os coeficientes do modelo que reduzem o desvio entre a saída observada e a prevista.

Com essa abordagem, conseguimos criar um modelo que retrata de forma apropriada o comportamento do amplificador, considerando tanto suas propriedades não lineares quanto seus efeitos de memória. A utilização do Python simplificou o processamento dos dados e a execução dos cálculos, possibilitando uma otimização eficaz dos coeficientes do modelo.

## 5.REFERÊNCIAS

"Método dos Mínimos Quadrados" - UEL. Disponível em  
<<https://www.uel.br/projetos/matessencial/superior/alinear/mmq.html#sec01>>

"Método dos Mínimos Quadrados" - UFRGS. Disponível em  
<[https://www.ufrgs.br/reatmat/AlgebraLinear/livro/s14-mx00e9todo\\_dos\\_mx00ednimos\\_quadradados.html](https://www.ufrgs.br/reatmat/AlgebraLinear/livro/s14-mx00e9todo_dos_mx00ednimos_quadradados.html)>

Polinômios com Memória de Complexidade Reduzida e sua Aplicação na Pré-distorção Digital de Amplificadores de Potência - Schuartz, Luis. Disponível em <<https://www.eletrica.ufpr.br/dokuwiki/arquivostccs/417.pdf>>

Referência Numpy. Disponível em  
<<https://numpy.org/doc/stable/reference/generated/numpy.linalg.lstsq.html>>

"Modelo de comportamiento basado en un polinomio con memoria". Disponível em  
<<https://biblus.us.es/bibing/proyectos/abreproy/11555/fichero/PFC+Joaqu%C3%ADn+Jos%C3%A9+Jim%C3%A9nez+Carreira+%252FCap%C3%ADtulo+6+-+Modelo+de+comportamiento+basado+en+un+polinomio+con+memoria.pdf>>